

FINAL DISCOUNTED PRICE

Company: Twitter

Round: Online Assessment

Year: 2021

Difficulty: Medium

Topic: Arrays, Stack, Monotonic Stack

Source: https://algo.monster/problems/twitter_oa_final_discounted_price

Problem Description:

You are given an array called prices. Each value represents the original price of an item. For every item, we look at the items after it and find the first item whose price is less than or equal to its price.

If we find such an item:

Discount = price of the first smaller or equal item

So the final price becomes:

Final Price = Original Price - Discount

If there is no such item after it, the item gets no discount.

Finally, we have to return the total money earned after all discounts.

Input Format:

The input is an integer array : prices where each element represents the price of an item.

Output Format:

Return an integer representing the total revenue after applying all possible discounts.

Sample Input:

prices = [2, 3, 1, 2]

Sample Output:

Total Revenue = 6

Explanation:

Let's go through the example:

[2, 3, 1, 2]

Item 0: Price = 2

Look at the items after it:

3, 1, 2

The first price that is less than or equal to 2 is:

1

So:

$$2 - 1 = 1$$

Final price = 1. Item

1: Price = 3

Look at:

1, 2

The first price less than or equal to 3 is 1.

So:

$$3 - 1 = 2$$

Final price = 2.

Item 2: Price = 1

The only item after it is:

2

2 is greater than 1, so it cannot be used as a discount.

Therefore, there is no discount.

Final price = 1.

Item 3: Price = 2 There

are no items after it.

Therefore, there is no discount.

Final price = 2.

So the final prices are:

[1, 2, 1, 2]

Total revenue: 1

+ 2 + 1 + 2 = 6

Therefore:

Answer = 6

Approach to Solve This Problem:

A simple approach would be to check every item against all the items after it. But that can be slow.

For example, if there are 100,000 items, repeatedly searching to the right could take $O(n^2)$ time.

Instead, we can use a Stack.

The important idea is:

We process the prices from right to left and use a stack to quickly find the first smaller or equal price.

Why Do We Go From Right to Left?

For each item, we only care about items after it.

So if we start from the right, all the items that an item needs to check have already been processed.

For example:

[2, 3, 1, 2]
 ↑

We start with the last item and move toward the beginning.

How the Stack Works?

The stack stores prices that may be useful as discounts.

For every current price:

Step 1: Remove bigger prices

If the top of the stack is greater than the current price, it cannot be the discount for the current item.

So we remove it. while (!stack.isEmpty() &&

stack.peek() > prices[i])

We continue removing bigger values until the top is smaller than or equal to the current price.

Step 2: Use the top as the discount

If the stack is not empty: discount

= stack.peek();

The top is the first suitable price we can use as a discount.

Step 3: Calculate the final price

Final Price = Current Price - Discount

If the stack is empty, there is no suitable discount.

Step 4: Add the current price

Finally, we put the current price into the stack:

stack.push(prices[i]);

It may become a discount for an item to its left.

Step-by-Step Example

Consider:

[2, 3, 1, 2]

We start from the right.

Price = 2

Stack is empty.

No discount.

Stack: [2]

Price = 1 Top

is 2.

Since: 2

> 1

remove

2.

The stack becomes empty, so 1 gets no discount.

Then add 1:

Stack: [1]

Price = 3 Top

is 1.

Since: $1 \leq 3$ we use 1

as the discount.

$3 - 1 = 2$

Then push 3.

Stack: [1, 3]

Price = 2

Top is 3.

Since: 3

> 2

remove

3.

Now the top is 1.

Since: $1 \leq 2$ use 1

as the discount.

$2 - 1 = 1$

Then push 2.

The final prices are:

[1, 2, 1, 2]

Total:

6

Java Code:

```
1  import java.util.*;
2
3  public class Main {
4
5      public static int finalDiscountedPrice(int[] prices) {
6
7          Stack<Integer> stack = new Stack<>();
8
9          int total = 0;
10
11         // Process from right to left
12         for (int i = prices.length - 1; i >= 0; i--) {
13
14             // Remove prices that are too large
15             while (!stack.isEmpty() && stack.peek() > prices[i]) {
16                 stack.pop();
17             }
18
19             // If a suitable discount exists
20             if (!stack.isEmpty()) {
21                 total += prices[i] - stack.peek();
22             } else {
23                 // No discount
24                 total += prices[i];
25             }
26
27             // Current price can be a discount
28             // for an item on its left
29             stack.push(prices[i]);
30         }
31
32         return total;
33     }
```

```
5  public static void main(String[] args) {
6
7      Scanner sc = new Scanner(System.in);
8
9      int n = sc.nextInt();
10
11     int[] prices = new int[n];
12
13     for (int i = 0; i < n; i++) {
14         prices[i] = sc.nextInt();
15     }
16
17     System.out.println(finalDiscountedPrice(prices));
18
19     sc.close();
20 }
21 }
```

Solution:

We use a **stack** and process the array from right to left.

For every price:

1. Remove prices from the stack that are greater than the current price.
2. If something remains, its top value is the discount.
3. Subtract the discount from the current price.
4. If nothing remains, there is no discount.
5. Push the current price into the stack.

This allows us to find the required discount efficiently without repeatedly searching through the array.

Understanding the Important Part of the Code:

1. Create the Stack

```
Stack<Integer> stack = new Stack<>();
```

The stack stores prices that can potentially be used as discounts.

2. Traverse from Right to Left

```
for (int i = prices.length - 1; i >= 0; i--  
)
```

We start from the last item because every item needs to look only at the items after it.

3. Remove Larger Prices while

```
(!stack.isEmpty() && stack.peek() >  
prices[i]) { stack.pop(); }
```

If the top price is greater than the current price, it cannot be used as a discount.

So we remove it.

4. Apply the Discount if

```
(!stack.isEmpty()) {  
    total += prices[i] - stack.peek(); }
```

If the stack isn't empty, its top contains the first suitable price.

We subtract it from the current price.

5. No Discount else

```
{  
    total += prices[i]; }
```


If the stack is empty, there is no smaller or equal price available.

So we add the original price.

6. Push the Current Price `stack.push(prices[i]);`

The current price may be useful as a discount for an item that comes before it.

Complexity Analysis:

Time Complexity: $O(n)$

Although there is a while loop inside the for loop, every price is pushed into the stack once and popped at most once.

Therefore, the total number of stack operations is proportional to n .

Time Complexity = $O(n)$

Space Complexity: $O(n)$

In the worst case, the stack can contain all n prices.

Space Complexity = $O(n)$